



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Course: Fundamentals of Data Science

Course Code: DSA0404

Slot: A

Faculty: KRISHNARAJ M

Submitted By

Y. Sravan Kumar (192472289)

Problem Solving Using Numpy, Pandas, Scipy, Scikit Learn Libraries in Python for Data Scientist

1. GENERATE RAW MOCK DATA (Simulating an imperfect CSV file)

Code:

```
employee_data.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/employee_data.py (3.12.3)
File Edit Format Run Options Window Help

import pandas as pd
import numpy as np

# 1. GENERATE RAW MOCK DATA (Simulating an imperfect CSV file)
raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000], # Missing value
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28",
    ],
}

# Create DataFrame
df = pd.DataFrame(raw_data)

# Display Raw Data
print("--- Raw Data ---")
print(df)
```

Output:

```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/employee_data.py
--- Raw Data ---
Employee_Name Department Salary Join_Date
0 Alice HR 50000.0 2022-01-15
1 Bob Engineering 85000.0 2021-06-20
2 Charlie HR NaN 2023-03-11
3 Diana Engineering 92000.0 2020-11-01
4 Evan Marketing 60000.0 2024-02-28
>>>
```

2. DATA PREPROCESSING

Code:

```
data_preprocessing.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/data_preprocessing.py (3.12.3)
File Edit Format Run Options Window Help
import pandas as pd
import numpy as np

# Employee Data
raw_data = {
    "Employee_Name": [" Alice ", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, np.nan, 92000, 60000],
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28"
    ]
}

df = pd.DataFrame(raw_data)

# 2. DATA PREPROCESSING

# Remove extra spaces from Employee_Name
df["Employee_Name"] = df["Employee_Name"].str.strip()

# Replace missing salary with the median salary
median_salary = df["Salary"].median()
df["Salary"] = df["Salary"].fillna(median_salary)

# Convert Join_Date to datetime format
df["Join_Date"] = pd.to_datetime(df["Join_Date"])

# Create Years_of_Service column
df["Years_of_Service"] = 2026 - df["Join_Date"].dt.year

print("--- Preprocessed Data ---")
print(df)
```

Output:

```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/data_preprocessing.py
--- Preprocessed Data ---
Employee_Name  Department  Salary  Join_Date  Years_of_Service
0      Alice      HR      50000.0  2022-01-15      4
1      Bob      Engineering  85000.0  2021-06-20      5
2      Charlie    HR      72500.0  2023-03-11      3
3      Diana    Engineering  92000.0  2020-11-01      6
4      Evan      Marketing  60000.0  2024-02-28      2
>>>
```

3. DATA AGGREGATION

Code:

```
DATA AGGREGATION.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/DATA AGGREGATION.py (3.12.3)
File Edit Format Run Options Window Help
import pandas as pd
import numpy as np

# Employee Data
raw_data = {
    "Employee_Name": ["Alice", "Bob", "Charlie", "Diana", "Evan"],
    "Department": ["HR", "Engineering", "HR", "Engineering", "Marketing"],
    "Salary": [50000, 85000, 72500, 92000, 60000],
    "Join_Date": [
        "2022-01-15",
        "2021-06-20",
        "2023-03-11",
        "2020-11-01",
        "2024-02-28"
    ]
}

df = pd.DataFrame(raw_data)

# 3. DATA AGGREGATION
# Group by Department and calculate average salary and total employees
dept_summary = (
    df.groupby("Department")
    .agg(
        Avg_Salary=("Salary", "mean"),
        Total_Employees=("Employee_Name", "count")
    )
    .reset_index()
)

print("--- Aggregated Summary ---")
print(dept_summary)
```

Output:

```
IDLE Shell 3.12.3
File Edit Shell Debug Options Window Help
Python 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/data_preprocessing.py
--- Preprocessed Data ---
Employee_Name  Department  Salary  Join_Date  Years_of_Service
0      Alice      HR      50000.0  2022-01-15      4
1      Bob      Engineering  85000.0  2021-06-20      5
2      Charlie    HR      72500.0  2023-03-11      3
3      Diana    Engineering  92000.0  2020-11-01      6
4      Evan      Marketing  60000.0  2024-02-28      2
>>>
```

4. DATA VISUALIZATION

Code:

```
DATA VISUALIZATION.py - C:/Users/Sravan Kumar.Y/AppData/Local/Programs/Python/Python312/DATA VISUALIZATION.py (3.12.3)
File Edit Format Run Options Window Help
)
    .reset_index()
)

# 4. DATA VISUALIZATION

# Set theme
sns.set_theme(style="whitegrid")

# Create two plots
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Plot 1: Bar Chart
sns.barplot(
    data=dept_summary,
    x="Department",
    y="Avg_Salary",
    hue="Department",
    palette="Blues_d",
    legend=False,
    ax=axes[0]
)

axes[0].set_title("Average Salary by Department")
axes[0].set_xlabel("Department")
axes[0].set_ylabel("Salary ($)")

# Plot 2: Scatter Plot
sns.scatterplot(
    data=df,
    x="Years_of_Service",
    y="Salary",
    hue="Department",
    style="Department",
    s=200,
    ax=axes[1]
)

axes[1].set_title("Salary vs Years of Service")
axes[1].set_xlabel("Years of Service")
axes[1].set_ylabel("Salary ($)")

plt.tight_layout()
plt.show()
```

Output:



